

1. Loop Statements in C#

C# provides several types of loop statements to control the flow of execution:

- **for loop:** Used when the number of iterations is known.
 - **foreach loop:** Iterates over a collection; read-only access.
 - **while loop:** Repeats a block while a condition is true.
 - **do-while loop:** Similar to while, but checks the condition **after** the first iteration.
 - **break:** Exits the loop immediately.
 - **continue:** Skips the current iteration and moves to the next.
-

2. Stack vs Heap in C#

Aspect	Stack	Heap
Purpose	Stores value types and function call frames	Stores reference types and dynamically allocated memory
Size	Limited (typically 1 MB per thread)	Dynamically growing
Speed	Fast (LIFO – Last In First Out)	Slower (requires GC)
Memory Management	Automatically managed via scope	Managed by the garbage collector
Lifetime	Short-lived (per method scope)	Long-lived (until GC collects)

Key Consideration: Stack is ideal for temporary data; heap is required for complex objects or data shared across scopes.

3. for vs foreach in C#

Feature	for Loop	foreach Loop
Usage	Index-based iteration	Collection-based iteration
Performance	Slightly faster (especially with arrays)	Easier to read; may be slower
Modifying Collection	Allowed (with caution)	Not allowed during iteration
Best For	Performance-critical loops	Clean, readable iteration

Summary: Use foreach for readability; use for when performance or index access is critical.

4. Nested for vs Single for: Performance Impact

- **Nested for loops** (e.g., for 2D arrays or combinations) **increase time complexity**, typically from $O(n)$ to $O(n^2)$, $O(n^3)$, etc.
- **Single for loop** is more efficient, with lower computational cost.

Example:

- Single loop: $O(n)$
- Nested two-level loop: $O(n^2)$

Recommendation: Avoid unnecessary nesting. Use nested loops only when logically required, and optimize with break/continue or algorithm redesign when possible.